

AI Stock Analysis & Recommendation Platform

Pratham Bhat | 2026-09-09

EDUCATIONAL USE ONLY — NOT FINANCIAL ADVICE. Do not use for real trading decisions.

1. Executive Summary

End-to-end stock analysis platform: CatBoost Sniper v5 predicts 10-trading-day direction using price, volume, VIX, and sentiment features. Dual-path sentiment: proxy at training time, live GDELT at inference. Multi-agent pipeline (ML + trend + risk) with horizon-weighted BUY/SELL/HOLD, inverse-vol position sizing, FastAPI, Gradio, Docker, MLflow, and CI.

2. System Architecture

Data: yfinance OHLCV, ^VIX, GDELT (inference), VADER/FinBERT. Training: sentiment proxy (sentiment_proxy.py) + 32 tickers × 10y, per-ticker chronological split. Model: CatBoost .cbm, 10 features, 10-day binary label. Serving: inference_pipeline.py → decision_agent.py → position_sizer.py → API/UI.

3. Held-Out Test Metrics (from sniper_metadata.json)

Threshold (val-tuned): 0.43 ROC-AUC: 0.5010271698728719 | Accuracy: 0.5375168236877523
Precision: 0.5376588403601784 | Recall: 0.9993743156577507 Train rows: 55368 | Sentiment mode: proxy

4. Overfitting Analysis — Methodology

How we check for overfitting (run after each training via overfitting_analysis.py): 1. Rebuild the dataset with identical settings (proxy sentiment, 10y, 32 tickers). 2. Apply per-ticker chronological 70/15/15 split — test set never seen during training. 3. Load the saved .cbm model; score train, validation, and test partitions. 4. Tune probability threshold on validation F1 only; apply same threshold to all splits. 5. Compare ROC-AUC across splits. Flag if train AUC – val AUC > 0.05 (overfitting risk). Additional safeguards during training: • CatBoost early stopping on validation AUC (best iteration retained, not max iterations). • Feature winsorization at 1st/99th percentile. • No random train/test shuffle (time-series safe).

5. Overfitting Analysis — Results

Verdict: possible_overfitting Train AUC (0.5727) exceeds validation AUC (0.5208) by 0.0518 — monitor generalization. AUC by split: Train: 0.5726723719545189 Validation: 0.5208310915574963 Test: 0.5010274545336282 Gaps: train–val AUC = 0.0518; val–test AUC = 0.0198 CatBoost best iteration: 103 (early stop vs 1200 max). Interpretation: Fast training (~2–10 min) is expected — CatBoost on ~55k tabular rows with early stopping is CPU-efficient. Speed alone does not imply overfitting; the val/test AUC gap does.

6. Limitations

- Modest predictive power (AUC ~0.50–0.52) — realistic for public retail features.
- Not a price-target or portfolio optimiser; directional research tool only.
- Long UI horizons are trend-dominated, not ML annual forecasts.
- EDUCATIONAL PURPOSE ONLY — NOT FINANCIAL ADVICE.

7. Key Code Snippets

sentiment_proxy.py

```

"""
Training-time sentiment without GDELT historical API calls.

GDELT rate-limits (HTTP 429) make multi-year backfill impractical locally.
Architecture:
    TRAINING → market-derived proxy + optional 1x yfinance news call per ticker
    INFERENCE → live GDELT (+ yfinance fallback) in sniper_predictor.py

```

```

Proxy formula (per row, backward-looking only):
    tanh(5d_return × k) × (1 + volume_zscore) clipped to [-1, 1]
"""

```

```

from __future__ import annotations

import logging

import numpy as np
import pandas as pd

from src.data.sentiment_scorer import score_headlines

logger = logging.getLogger(__name__)

def _yfinance_headline_score(ticker: str) -> float | None:

```

Per-ticker split

```

"""Train/val/test splitting utilities (no heavy data dependencies)."""

from __future__ import annotations

import logging

import pandas as pd

logger = logging.getLogger(__name__)

def chronological_split(
    df: pd.DataFrame,
    train_ratio: float = 0.70,
    val_ratio: float = 0.15,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """Global time-ordered split."""
    n = len(df)
    train_end = int(n * train_ratio)
    val_end = int(n * (train_ratio + val_ratio))

    train_df = df.iloc[:train_end].copy()
    val_df = df.iloc[train_end:val_end].copy()
    test_df = df.iloc[val_end:].copy()

    if len(train_df) == 0 or len(val_df) == 0 or len(test_df) == 0:
        raise ValueError("Split produced an empty partition; use more data or adjust ratios.")

    return train_df, val_df, test_df

def per_ticker_chronological_split(
    df: pd.DataFrame,
    train_ratio: float = 0.70,
    val_ratio: float = 0.15,

```

Decision blend

```

trend_prob = (trend_score + 1.0) / 2.0
composite = ml_w * ml_score + trend_w * trend_prob

if composite >= BUY_THRESHOLD:
    decision = "BUY"
    confidence = composite
elif composite <= SELL_THRESHOLD:
    decision = "SELL"
    confidence = 1.0 - composite
else:
    decision = "HOLD"
    confidence = max(composite, 1.0 - composite)

# Risk overlay: downgrade BUY in extreme volatility
risk_adjusted = decision

```

```

risk_note = ""
if risk_overlay and decision == "BUY":
    if risk_overlay.get("risk_level") == "HIGH":
        risk_adjusted = "HOLD"
        risk_note = (
            " BUY downgraded to HOLD due to elevated volatility "
            f"(risk score {risk_overlay.get('risk_score', 'N/A')})."
        )

```

Early stopping params

```

# CatBoost Sniper v5 defaults (reproducible training)
SNIPER_CATBOOST_PARAMS = {
    "iterations": 1200,
    "learning_rate": 0.02,
    "depth": 6,
    "l2_leaf_reg": 12,
    "random_seed": 42,
    "verbose": 100,
    "eval_metric": "AUC",
    "auto_class_weights": "Balanced",
    "early_stopping_rounds": 80,
}

SNIPER_CONF_THRESHOLD = 0.50 # fallback; trainer tunes on validation F1

```

8. Verification: PASSED